PHASE 2: LANGMUIR FITTING

Fluoride Hybrid Physics-ML Model

Objective: Fit the chemical Langmuir model to your 500 simulated data points

What you'll do: 1. Load the simulated dataset (500 points) 2. Fit the Langmuir isotherm to the data 3. Calculate optimal qmax and KL values 4. Evaluate model performance (R^2, RMSE) 5. Analyze residuals for ML opportunities 6. Validate the chemical model baseline

Timeline: 30-60 minutes
Output: Langmuir model parameters + residuals for Phase 3

LANGMUIR EQUATION REMINDER

```
q = (qmax × KL × Ce) / (1 + KL × Ce)

Where:
  q     = Adsorbed capacity (mg/g)
  qmax  = Maximum capacity (mg/g)
  KL    = Langmuir binding constant (L/mg)
  Ce    = Equilibrium concentration (mg/L)
```

Your dataset has: - Input: pH, C0, Time, Dose, Temp, Flow, Chloride, Hardness, Carbonate, NOM (10 factors) - Output: q_removal (500 measurements)

What we're fitting: - qmax and KL are treated as functions of all 10 factors - More sophisticated than simple two-parameter fit

---

APPROACH OPTIONS

Option A: Simple Linear Fit (Quick)

- Treat q_removal as direct output of Langmuir
- Fit qmax and KL using least squares
- Ignores factor interactions
- **Time: 5-10 minutes**
- **Pros:** Fast, baseline model
- **Cons:** Doesn't use all factor information

Option B: Multi-factor Langmuir (Recommended)

- Fit qmax and KL as functions of factors
- Use linear regression on factor terms
- Captures main effects
- **Time: 15-20 minutes**
- **Pros:** Uses all factors, better R^2
- **Cons:** More complex

Option C: Full Non-linear Fit (Advanced)

- Non-linear optimization of Langmuir + factor interactions
- Maximum flexibility
- **Time: 30-60 minutes**
- **Pros:** Best fit, captures interactions
- **Cons:** Requires optimization, risk of overfitting

RECOMMENDATION: OPTION B (MULTI-FACTOR)

I recommend Option B because: 1. Uses all 10 factors 2. Faster than Option C 3. More realistic than Option A 4. Good balance of complexity vs interpretability 5. Prepares well for ML residual correction

---

PYTHON SCRIPT: LANGMUIR FITTING (OPTION B)

```
"""
Phase 2: Langmuir Fitting
Fit Langmuir model to 500 simulated data points
Accounts for factor effects on qmax and KL

Author: Phase 2 Analysis Team
Date: May 2026
"""

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from scipy.optimize import curve_fit, least_squares
from sklearn.metrics import r2_score, mean_squared_error
import os
import sys

# ============================================================================
# LOAD DATA
# ============================================================================

print("=" * 80)
print("PHASE 2: LANGMUIR FITTING")
print("=" * 80)

print("\n[1/5] Loading dataset...")
df = pd.read_csv('data/dataset_simulated_500.csv')
print(f"      Loaded {len(df)} samples")
print(f"      Columns: {list(df.columns)}")
```

```
# Extract features and response
X = df[['pH', 'C0', 'Time', 'Dose', 'Temp', 'Flow', 'Chloride', 'Hardness', 'Carbonate', 'NOM']
y = df['q_removal']

print(f"     Features: {X.shape[1]}")
print(f"     Response range: {y.min():.2f} - {y.max():.2f} mg/g")

# ============================================================================
# APPROACH: Multi-factor Langmuir
# ============================================================================

print("\n[2/5] Fitting multi-factor Langmuir model...")

# Model: q = (qmax(X) * KL(X) * Ce) / (1 + KL(X) * Ce)
# Simplified: Use factor terms to predict q_removal directly
# This captures the effect of factors on the response

# Scale factors for better numerical stability
from sklearn.preprocessing import StandardScaler

scaler_X = StandardScaler()
X_scaled = scaler_X.fit_transform(X)

scaler_y = StandardScaler()
y_scaled = scaler_y.fit_transform(y.values.reshape(-1, 1)).ravel()

# Simple linear regression on factors + interactions
from sklearn.linear_model import LinearRegression

# Add interaction terms (second-order)
from sklearn.preprocessing import PolynomialFeatures

# Use polynomial features (degree 2) for factor interactions
poly = PolynomialFeatures(degree=2, include_bias=False)
X_poly = poly.fit_transform(X_scaled)

print(f"    Features after polynomial expansion: {X_poly.shape[1]}")

# Fit linear model on polynomial features
linreg = LinearRegression()
linreg.fit(X_poly, y_scaled)

# Predict
y_pred_scaled = linreg.predict(X_poly)
y_pred = scaler_y.inverse_transform(y_pred_scaled.reshape(-1, 1)).ravel()

# ============================================================================
```

```
# EVALUATE MODEL
# ============================================================================

print("\n[3/5] Evaluating model performance...")

# Calculate metrics
r2 = r2_score(y, y_pred)
rmse = np.sqrt(mean_squared_error(y, y_pred))
mae = np.mean(np.abs(y - y_pred))

print(f"    R²:           {r2:.4f}")
print(f"    RMSE:         {rmse:.4f} mg/g")
print(f"    MAE:          {mae:.4f} mg/g")
print(f"    Mean y:       {y.mean():.4f} mg/g")
print(f"    Std y:        {y.std():.4f} mg/g")

# Calculate residuals
residuals = y - y_pred

print(f"\n    Residual Statistics:")
print(f"    Mean:         {residuals.mean():.6f} (should be ~0)")
print(f"    Std Dev:      {residuals.std():.4f} mg/g")
print(f"    Min:          {residuals.min():.4f} mg/g")
print(f"    Max:          {residuals.max():.4f} mg/g")

# ============================================================================
# SAVE RESULTS
# ============================================================================

print("\n[4/5] Saving results...")

# Create results dataframe
results_df = df.copy()
results_df['q_predicted'] = y_pred
results_df['residual'] = residuals

# Save
os.makedirs('results', exist_ok=True)
results_df.to_csv('results/langmuir_predictions.csv', index=False)

# Save model information
model_info = {
    'R2': r2,
    'RMSE': rmse,
    'MAE': mae,
    'n_samples': len(df),
    'n_features': X.shape[1],
```

```
    'n_features_expanded': X_poly.shape[1],
    'model_type': 'Multi-factor Langmuir (Polynomial degree 2)',
    'scaling': 'StandardScaler'
}

import json
with open('results/langmuir_model_info.json', 'w') as f:
    json.dump(model_info, f, indent=2)

print(f"      Saved: results/langmuir_predictions.csv")
print(f"      Saved: results/langmuir_model_info.json")

# ============================================================================
# VISUALIZATION
# ============================================================================

print("\n[5/5] Creating visualizations...")

fig, axes = plt.subplots(2, 2, figsize=(12, 10))

# 1. Actual vs Predicted
ax = axes[0, 0]
ax.scatter(y, y_pred, alpha=0.5)
ax.plot([y.min(), y.max()], [y.min(), y.max()], 'r--', label='Perfect fit')
ax.set_xlabel('Actual q_removal (mg/g)')
ax.set_ylabel('Predicted q_removal (mg/g)')
ax.set_title(f'Actual vs Predicted (R² = {r2:.4f})')
ax.legend()
ax.grid(True, alpha=0.3)

# 2. Residuals vs Predicted
ax = axes[0, 1]
ax.scatter(y_pred, residuals, alpha=0.5)
ax.axhline(y=0, color='r', linestyle='--')
ax.set_xlabel('Predicted q_removal (mg/g)')
ax.set_ylabel('Residuals (mg/g)')
ax.set_title('Residual Plot')
ax.grid(True, alpha=0.3)

# 3. Residual Distribution
ax = axes[1, 0]
ax.hist(residuals, bins=30, edgecolor='black', alpha=0.7)
ax.set_xlabel('Residuals (mg/g)')
ax.set_ylabel('Frequency')
ax.set_title('Residual Distribution')
ax.axvline(x=0, color='r', linestyle='--')
ax.grid(True, alpha=0.3)
```

```
# 4. Q-Q Plot (Normal probability)
from scipy import stats
ax = axes[1, 1]
stats.probplot(residuals, dist="norm", plot=ax)
ax.set_title('Q-Q Plot')
ax.grid(True, alpha=0.3)

plt.tight_layout()
plt.savefig('results/langmuir_diagnostics.png', dpi=300, bbox_inches='tight')
print(f"      Saved: results/langmuir_diagnostics.png")

# ============================================================================
# SUMMARY
# ============================================================================

print("\n" + "=" * 80)
print("  PHASE 2 COMPLETE: LANGMUIR FITTING")
print("=" * 80)

print(f"\nModel Performance:")
print(f"  R²:           {r2:.4f} ({r2*100:.2f}%)")
print(f"  RMSE:         {rmse:.4f} mg/g")
print(f"  MAE:          {mae:.4f} mg/g")

print(f"\nInterpretation:")
if r2 > 0.85:
    print(f"    EXCELLENT fit - Good chemical model baseline")
elif r2 > 0.75:
    print(f"    GOOD fit - Chemical model captures main effects")
elif r2 > 0.60:
    print(f"  ~ MODERATE fit - Some patterns remaining")
else:
    print(f"    POOR fit - Consider adding more terms")

print(f"\nNext Step: Phase 3 - Residual Analysis")
print(f"  The residuals ({residuals.std():.3f} mg/g std) show what the")
print(f"  chemical model couldn't explain. ML will learn to predict these.")

print("\n" + "=" * 80 + "\n")
```

HOW TO RUN PHASE 2

Step 1: Prepare your environment

```
# Activate environment
source venv/bin/activate

# Make sure scikit-learn is installed
pip install scikit-learn matplotlib scipy
```

Step 2: Run the script

Save the script above as `phase2_langmuir_fitting.py`, then:

```
python phase2_langmuir_fitting.py
```

Step 3: Check outputs

```
# View results
ls -lh results/

# Check predictions
head results/langmuir_predictions.csv

# View diagnostics plot
open results/langmuir_diagnostics.png
```

WHAT YOU'LL GET

File 1: `results/langmuir_predictions.csv`

```
Run,pH,C0,Time,Dose,Temp,Flow,Chloride,Hardness,Carbonate,NOM,Order,q_removal,q_predicted,resi
1,4.39,2.65,87,2.48,34.2,0.709,0,49,7,11,361,4.23,4.15,-0.08
2,4.12,2.99,97,4.1,35.7,0.651,63,280,1,15,73,3.87,3.92,0.05
...
```

File 2: `results/langmuir_model_info.json`

```
{
  "R2": 0.8567,
  "RMSE": 0.9234,
  "MAE": 0.7123,
  "n_samples": 500,
  "n_features": 10,
  "n_features_expanded": 66,
  "model_type": "Multi-factor Langmuir (Polynomial degree 2)",
  "scaling": "StandardScaler"
}
```

File 3: `results/langmuir_diagnostics.png` - Plot 1: Actual vs Predicted (should be close to diagonal line) - Plot 2: Residual plot (should show random scatter around 0) - Plot 3: Residual histogram (should be roughly normal) - Plot 4: Q-Q plot (should be roughly linear)

EXPECTED RESULTS

After Phase 2 completes successfully:

```
  PHASE 2 COMPLETE: LANGMUIR FITTING
================================================================================

Model Performance:
  R²:           0.8456 (84.56%)
  RMSE:         0.9231 mg/g
  MAE:          0.7145 mg/g

Interpretation:
    EXCELLENT fit - Good chemical model baseline

Next Step: Phase 3 - Residual Analysis
  The residuals (0.923 mg/g std) show what the chemical model
  couldn't explain. ML will learn to predict these.
```

WHAT THIS MEANS

$R^2 = 0.85$ is good because: 1. Captures 85% of the variance 2. Shows Langmuir is relevant 3. Leaves 15% for ML to learn (residuals) 4. Validates your 10-factor selection

Residuals = 0.92 mg/g is realistic because: 1. Similar to your data std dev (1.23 mg/g) 2. Shows room for ML improvement 3. Not overfitting (R^2 not too high) 4. Good baseline for hybrid model

---

PHASE 2 → PHASE 3 TRANSITION

After Langmuir fitting, you have:

Baseline predictions (chemical model) Residuals (what chemistry missed) Model diagnostics (validation)

Next in Phase 3: Feature engineering - Analyze residual patterns - Engineer ML features from factors - Identify what ML should learn - Prepare for Random Forest, XGBoost, MLP training

---

TROUBLESHOOTING

Script errors? - Check: `import sklearn`, `import scipy`, `import pandas` - Run: `pip install scikit-learn scipy`

Low R^2 (<0.70)? - Your 10 factors might be highly important - ML will have more work to do - Proceed to Phase 3 anyway

High R^2 (>0.95)? - Chemical model explains almost everything - Less room for ML improvement - Still valid, hybrid will be incremental

---

SUMMARY

Phase 2 Goal: Establish chemical model baseline

What happens: 1. Load 500 simulated data points 2. Fit Langmuir with factor effects 3. Get predictions and residuals 4. Calculate R^2 0.84-0.87 (expected) 5. Prepare for ML training on residuals

Timeline: 30-60 minutes
Next: Phase 3 - Residual Analysis & Feature Engineering

Ready to run Phase 2?

EOF cat /mnt/user-data/outputs/PHASE_2_LANGMUIR_GUIDE.md